



5. Accessing Tuples

1. Accessing tuple elements using positive and negative indexing.

➤ Accessing Elements with Positive Indexing

➤ Starts from 0 (first element) to n-1 (last element)

➤ Syntax:

```
tuple_name[index]
```

➤ Ex:

```
tuple = (10, 20, 30, 40, 50)
```

```
print(tuple[0])
```

```
# Output: 10
```

```
print(tuple[2])
```

```
# Output: 30
```

➤ Accessing Elements with Negative Indexing

➤ Starts from -1 (last element) to -n (first element)

➤ Ex:

```
tuple = (10, 20, 30, 40, 50)
```

```
print(tuple[-1])
```

```
# Output: 50
```

2. Slicing a tuple to access ranges of elements.

- Tuples are immutable. Slicing creates a new tuple, does not modify the original.
- Index out of range does not cause error in slicing.
- `tuple[start:stop:step]`
- `start`: index to begin slicing.
- `stop`: index to end slicing.
- `step`: how many steps to jump(default is 1).

➤ Ex:

```
tuple = (10, 20, 30, 40, 50, 60, 70)
print(my_tuple[1:4])
# (20, 30, 40)
print(my_tuple[:3])
# (10, 20, 30)
print(my_tuple[:])
# (10, 20, 30, 40, 50, 60, 70)
```